

Create a job -- select Pipeline --> under script --> use below or --> try sample pipeline and explain

short cut to remember how to create a pipeline :

PASSS - PAS3
Pipeline,
Agent,
Stages,
Stage ,
Steps

Below pipeline has one stage

```
pipeline {
    agent any

    stages {
        stage('one') {
            steps {
                sh 'touch file1'
            }
        }
    }
}
```

--> now, file1 is created at cd /var/lib/jenkins/workspace/

Can have single stage or multi stage, depend upon requirement

```
pipeline {
    agent any
    stages {
        stage('one'){
            steps {
                sh 'touch file1'
            }
        }
        stage('two'){
            steps {
                sh 'ls -al'
            }
        }
        stage('three'){
            steps{
                sh 'df -h'
            }
        }
    }
}
```

=====
Now create a CI Pipeline
=====

CODE --> BUILD --> TEST -->Code Review --> ARTIFACT --> Deployment

```
pipeline {
    agent any
    stages {
```

```

stage('Clone Git Repo') {
    steps {
        echo 'Cloning Git Repository'
    }
}
stage('Building') {
    steps {
        echo 'Building the Project'
    }
}
stage('Code Test') {
    steps {
        echo 'Testing the Project'
    }
}
stage('Code Review') {
    steps {
        echo 'Code Quality Checking'
    }
}
stage('Artifact') {
    steps {
        echo 'Generating war file'
    }
}
stage('Deployment') {
    steps {
        echo 'Code is deployed'
    }
}
}
}

```

```

pipeline {
    agent any
    stages {
        stage('checkout') {
            steps {
                git 'https://github.com/ReyazShaik/java-project-maven-
new.git'
            }
        }
        stage('compile') {
            steps {
                sh 'mvn compile'
            }
        }
        stage('test') {
            steps {
                sh 'mvn test'
            }
        }
        stage('artifact') {
            steps {
                sh 'mvn clean package'
            }
        }
        stage('Deployment') {
            steps {

```

```

        echo 'Code is deploy - actual we need to learn tomcat, using
tomcat we deploy'
    }
}
}

```

or

Pipeline as a Code - Running more than one command inside a single stage
(Advantages: save time, can run all commands in one stage, reduce length of code)

=====

```

pipeline {
    agent any
    stages {
        stage('one') {
            steps {
                git 'https://github.com/ReyazShaik/java-project-maven-
new.git#39;
                sh 'mvn compile'
                sh 'mvn test'
                sh 'mvn package'
            }
        }
    }
}

```

Multi stage pipeline as a Code

=====

```

pipeline {
    agent any

    stages {
        stage('one') {
            steps {
                git 'https://github.com/ReyazShaik/java-project-maven.git#39;
                sh 'mvn compile'
            }
        }

        stage('two') {
            steps {
                sh 'mvn test'
                sh 'mvn package'
            }
        }
    }
}

```

Pipeline as a Code over single shell

=====

```

pipeline {
    agent any
    stages {

```

```

        stage('checkout') {
            steps {
                git 'https://github.com/ReyazShaik/java-project-maven-
new.git#39;
                sh '''
                sh mvn compile
                sh mvn test
                sh mvn clean package
                '''
            }
        }
    }
}

```

=====

INPUT PARAMETERS - Manual input : In real time, we use input to continue the pipeline like yes or no

=====

Create a new pipeline for input parameter and copy the below code

```

pipeline {
    agent any
    stages {
        stage ('checkout') {
            steps {
                git 'https://github.com/ReyazShaik/java-project-maven-
new.git#39;
            }
        }
        stage ('build') {
            steps {
                sh 'mvn compile'
            }
        }
        stage ('test') {
            steps {
                sh 'mvn test'
            }
        }
        stage ('artifact') {
            steps {
                sh 'mvn package'
            }
        }
        stage ('deploy') {
            input {
                message "is your inputs correct ?"
                ok "yes"
            }
            steps {
                echo "my code is deployed"
            }
        }
    }
}

```

=====

Linked Jobs - Link the jobs with each other --> first job will trigger another job

=====

Create job1 and job2 with build steps --> execute shell --> echo "welcome"

Upstream and downstream

job1 - Post-build Actions -->Build other projects--> Projects to build --> job2

Build the job now, and once first job build done and second job will get trigger

=====

MASTER and SLAVE

=====

MASTER AND SLAVE:

when we continuously build jobs on Jenkins ,it will get load and have the chance to crash.

to avoid this load on Jenkins we use slave servers to build the jobs

Setup Master

=====

Install Jenkins with Script

SETUP: SLAVE 1

CREATE AN EC2 INSTANCE Amazon Linux 2 AND INSTALL JAVA-11/17

amazon-linux-extras install java-openjdk11 -y

or

sudo yum install java-17-amazon-corretto-devel maven git -y

Dashboard -- > Manage Jenkins -- >Nodes -- > New node-- > name: slave1 -- > Permanent Agent -- > create

Number of executors : 3 (number of parallel build we can do)

Remote root directory : /tmp (where your op is going to store)

Labels : slave1 (way of assigning work to particular slave)

Usage : last opt

Launch method : last opt

Host : private-ip of slave

Credentials : add -- > Jenkins -- >

kind: ssh username with private key

Username: ec2-user

Private Key: enter directly -- > copy paste the pem content

Host Key Verification Strategy: last opt

=====

Now Setup Slave 2

Follow above process -- but one thing generally miss - install amazon-linux-extras install java-openjdk11 -y on slave 2 first and then add node in Jenkins

Create a job , execute on build

if build fails --> on slave see java version

update-alternatives --config java

run the build again

--> if you want to run the pipeline using slave not in configuring Jenkins(Restrict where this project can be run), but directly through pipeline use the below script as agent label

Example 2 with Pipeline

```
pipeline {
    agent {
        label 'slave1'
    }

    stages {
        stage('checkout') {
            steps {
                git 'https://github.com/ReyazShaik/java-project-maven-
new.git';
                (or)
                git branch: 'main', url: 'https://github.com/ReyazShaik/java-project-
maven.git';
            }
        }
        stage('compile') {
            steps {
                sh 'mvn compile'
            }
        }
        stage('test') {
            steps {
                sh 'mvn test'
            }
        }
        stage('artifact') {
            steps {
                sh 'mvn clean package'
            }
        }
        stage('Deployment') {
            steps {
                echo 'Code is deploy - actual we need to learn tomcat, using
tomcat we deploy'
            }
        }
    }
}
```

=====

Monitor Slave machines --> Click on Node --> left side --> Load statistics, System info , Build history etc